
Oliver Krauss

Data Structures II: Trees & Maps

ALD | MTD.ba VZ SS26

Hagenberg 07.04.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Where We Left Off

Previously we built linear structures from scratch -- today we branch out into hierarchical data.

You previously implemented Stack, Queue, and LinkedList using a plain **Node** with **Object value** and a single **next** pointer:

Recap: Where We Left Off

Previously we built linear structures from scratch -- today we branch out into hierarchical data.

You previously implemented Stack, Queue, and LinkedList using a plain **Node** with **Object value** and a single **next** pointer:

- **Stack**: LIFO -- **push/pop/peek**, array-backed with **top** index
- **Queue**: FIFO -- **offer/poll**, two pointers **head/tail**
- **LinkedList**: **Node** chain -- $O(1)$ front/back insert, $O(n)$ random access
- **Recursion on lists**: traverse with **while(current != null)**

Today: give each **Node** two **next** pointers (**left** and **right**) -- and you have a binary tree.

Today's Roadmap

Topics

1. **Binary Trees** -- terminology, structure, height
2. **Tree Traversals** -- inorder, preorder, postorder, BFS
3. **Binary Search Tree** -- insert, search, BST property
4. **Java Maps & Sets** -- HashMap, TreeMap, hashCode/equals
5. **(optional) Graphs:** cycles, adjacency matrix/list, BFS/DFS with **int** vertices and arrays (stack/queue from

Why this matters

- File systems, game scene graphs, parse trees -- all are trees
- **HashMap** is the most-used collection in Java -- you need to know its contract
- BST insert uses the same recursive pattern as LinkedList traversal -- you already know it
- Optional graphs use the **same** queue and stack ideas---without generics; Session 7 adds reusable collection types

Tree vs. List/Array

When lists break down

- **File system**: a folder contains *subfolders* -- not a flat list
- **Game scene graph**: a `GameWorld` contains `Rooms`, each contains `Enemies`
- **Expression parser**: $(2+3)*4$ becomes a tree of operators and operands
- **HTML DOM**: `<body>` contains `<div>`s, which contain `<p>`s

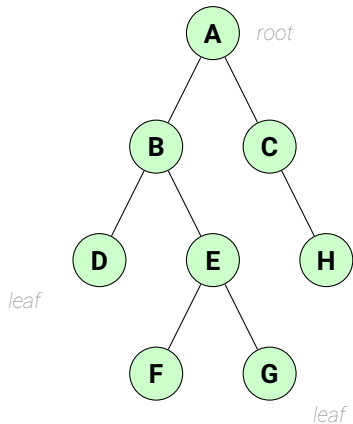
Key insight

A `LinkedList` is **linear** -- each node has one `next`.

A **binary tree** node has two children: `left` and `right`.

This single change unlocks hierarchical structure and $O(\log n)$ search.

Tree Terminology



- **Root**: topmost node (no parent) -- node A
- **Leaf**: node with no children -- D, F, G, H
- **Edge**: connection between parent and child
- **Height**: longest root-to-leaf path = 3
- **Depth**: distance from root (root has depth 0)
- **Subtree**: any node + all its descendants

TreeNode -- The Building Block

```
1  public class TreeNode {
2      // public for simplicity -- production
      code uses getters/setters
3  public Object value;
4  public TreeNode left;
5  public TreeNode right;
6
7  public TreeNode(Object value) {
8      this.value = value;
9      // left and right are null by
      default
10 }
11 }
```

Contrast with LinkedList (S5)

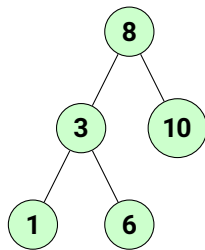
```
1  // LinkedList node (S5):
2  class Node {
3      Object value;
4      Node next;    // one pointer
5  }
6
7  // Tree node:
8  class TreeNode {
9      Object value;
10     TreeNode left;    // two
11     TreeNode right;   // pointers
12 }
```

Same pattern!

Just two **next** pointers instead of one.

Building a Tree Manually

```
1  // Each line adds one node to the tree:
2  TreeNode root  = new TreeNode(8);
3  root.left      = new TreeNode(3);
4  root.right     = new TreeNode(10);
5  root.left.left = new TreeNode(1);
6  root.left.right = new TreeNode(6);
```



Each line of code = one arrow in the diagram

`root.left = new TreeNode(3)` creates node 3 and wires it as the left child of 8.

Recursion on Trees -- The Universal Pattern

Almost every tree method looks like this

1. **Base case:** if (node == null)
return *neutral*
2. **Left subtree:** recurse on `node.left`
3. **Right subtree:** recurse on `node.right`
4. **Combine:** merge both results
(problem-specific)

Template (pseudocode)

```
result solve(node):  
    if node == null: return neutral  
    left = solve(node.left)  
    right = solve(node.right)  
    return combine(left, right)
```

Same skeleton everywhere

Different algorithms = different *neutral* and *combine*.

E.g. `height()`: *neutral* = -1,
combine = 1 + max(left, right)

From linked lists to trees

You already know the idea

In S5 you walked a list recursively:

```
if (node == null) return;  
// do work on this node  
recurse(node.next);
```

A binary tree is the same story with **two** outgoing edges instead of one: recurse to **left**, then recurse to **right**.

One pattern, many algorithms

Inorder, preorder, postorder, **height()**, **size()** - same four-step shape; only the “work” and *combine* step change.

height() -- Recursive

```
1 public class BinaryTree {
2     private TreeNode root;
3
4     /** Height = longest root-to-leaf path */
5     public int height() {
6         return height(root);
7     }
8
9     private int height(TreeNode node) {
10         if (node == null) return -1; // base
11                                     case
12         int leftH = height(node.left);
13         int rightH = height(node.right);
14         return 1 + Math.max(leftH, rightH);
15     }
16 }
```

Recursion pattern

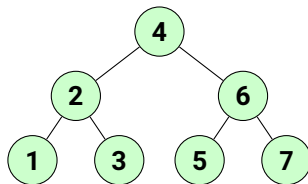
- **Base case:** null node returns -1
- **Recursive step:** compute left and right heights, take $\max + 1$
- **Every node visited:** $O(n)$
- **Two methods:** public API hides the node parameter; private helper carries the node as it recurses
- **Why -1 for null?** A single leaf node should have height 0 .
 $\text{height(leaf)} = 1 + \max(-1, -1) = 0 \checkmark$

Three Orderings, One Tree

The three DFS (Depth-First Search) traversals

DFS = go as deep as possible before backtracking.

Name	Order	Mnemonic
Inorder	Left, Root, Right	<i>In</i> = left → right
Preorder	Root, Left, Right	<i>Pre</i> = root first (copy)
Postorder	Left, Right, Root	<i>Post</i> = root last (delete)



Level-order (BFS = Breadth-First Search)

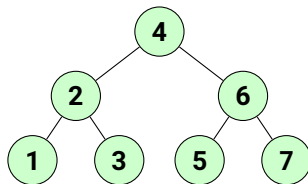
Visit nodes level by level, left to right. Uses a **Queue**

Three Orderings, One Tree

The three DFS (Depth-First Search) traversals

DFS = go as deep as possible before backtracking.

Name	Order	Mnemonic
Inorder	Left, Root, Right	<i>In</i> = left → right
Preorder	Root, Left, Right	<i>Pre</i> = root first (copy)
Postorder	Left, Right, Root	<i>Post</i> = root last (delete)



-- Inorder: 1 2 3 **4** 5 6 7

-- Preorder: **4** 2 1 3 6 5 7

-- Postorder: 1 3 2 5 7 6 **4**

Level-order (BFS = Breadth-First Search)

Visit nodes level by level, left to right. Uses a **Queue**

Inorder Traversal

```
1  /** Inorder: Left -> Root -> Right */
2  public void inorder(TreeNode node) {
3      if (node == null) return; // base case
4      inorder(node.left);       // 1. left
5      IO.print(node.value + " "); // 2. root
6      inorder(node.right);      // 3. right
7  }
```

Key property

For a **Binary Search Tree**, inorder traversal always yields elements in **sorted ascending order**.

Use cases

- Print BST elements sorted
- Validate BST property
- Generate sorted output

Preorder Traversal

```
1  /** Preorder: Root -> Left -> Right */
2  public void preorder(TreeNode node) {
3      if (node == null) return; // base case
4      IO.print(node.value + " "); // 1. root
5      preorder(node.left);       // 2. left
6      preorder(node.right);      // 3. right
7  }
```

Use cases

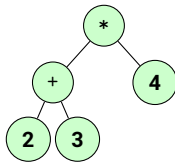
- **Serialize/copy** a tree -- root before children, so you can rebuild it
- Print directory listings (**ls -R** style)

Postorder Traversal

```
1  /** Postorder: Left -> Right -> Root */
2  public void postorder(TreeNode node) {
3      if (node == null) return; // base case
4      postorder(node.left);      // 1. left
5      postorder(node.right);     // 2. right
6      IO.print(node.value + " "); // 3. root
7  }
```

Use cases

- **Delete** a tree safely -- children freed before parent
- **Evaluate** expression trees:



Postorder visits leaves before operators; prints

2 3 + 4 *.

Level-order Traversal (BFS)

```
1  // Level-order: visit by depth, left to
   right
2  public void levelOrder(TreeNode root) {
3      if (root == null) return;
4      Queue queue = new Queue();
5      queue.offer(root);
6
7      while (!queue.isEmpty()) {
8          TreeNode node = queue.poll();
9          IO.print(node.value + " ");
10         if (node.left != null) queue.offer(
11             node.left);
12         if (node.right != null) queue.offer(
13             node.right);
14     }
15 }
```

The Queue drives level-order traversal:

- Enqueue root
- Poll a node, print it
- Enqueue its children
- Repeat until empty

java.util.Queue is the stdlib interface -- S7 covers the full Collections hierarchy.

Output for the 7-node tree

4 2 6 1 3 5 7
(level by level)

Iterative “go deep, then backtrack”?

Can we use a **while** loop instead of recursion?

Often yes - but for traversals that go **down a branch before returning** (inorder, preorder, postorder), you usually need an **explicit stack**: you push and pop **TreeNode** references yourself to remember where to resume. That is hand-coding what the JVM call stack does automatically when you recurse.

Linked list

One **next** pointer $\Rightarrow$ a plain loop walks the whole structure:

```
while (node != null) { ...; node =  
node.next; }
```

Binary tree

Two children $\Rightarrow$ after one subtree you must **backtrack** to the parent and then visit the other side. A bare **while** without a stack cannot express that; recursion (or an explicit stack) can.

height() recursion vs. iteration

Recursive

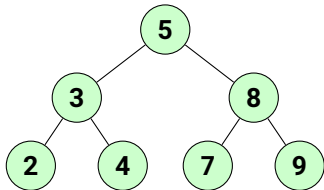
```
1 private int height(TreeNode node
    ) {
2     if (node == null) return -1;
3     int hL = height(node.left);
4     int hR = height(node.right);
5     return 1 + Math.max(hL, hR);
6 }
```

Iterative: queue, count levels

```
1 public int height(TreeNode root) {
2     if (root == null) return -1;
3     LinkedList q = new LinkedList();
4     q.add(root);
5     int h = -1;
6     while (!q.isEmpty()) {
7         int levelSize = q.size();
8         h++;
9         for (int i = 0; i < levelSize; i++) {
10             TreeNode node = (TreeNode) q.removeFirst();
11             if (node.left != null) q.add(node.left);
12             if (node.right != null) q.add(node.right);
13         }
14     }
15     return h;
16 }
```

Same longest root-to-leaf path: each outer loop drains one level of the tree (level-order pattern).

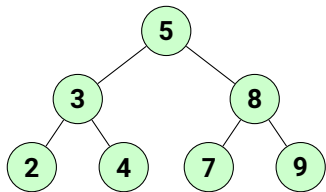
Interactive Break #1 -- Trace the Traversal



Fill in the traversal results for this 7-node tree:

Traversal	Result
Inorder	
Preorder	
Postorder	
Level-order	

Interactive Break #1 -- Answer



Traversal	Result
Inorder	2 3 4 5 7 8 9
Preorder	5 3 2 4 8 7 9
Postorder	2 4 3 7 9 8 5
Level-order	5 3 8 2 4 7 9

Key observation

Inorder of a BST = sorted order. This tree satisfies the BST property: every left child $<$ parent $<$ right child.

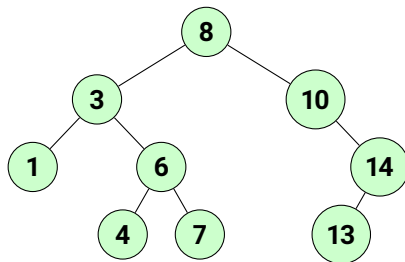
The Binary Search Tree (BST) Property

BST invariant

For every node **N**:

- All values in the **left** subtree are $< \mathbf{N.value}$
- All values in the **right** subtree are $\geq \mathbf{N.value}$

This ordering enables **binary search**: at each node, go left or right to halve the search space.



BST -- insert()

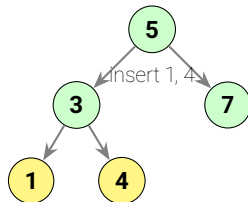
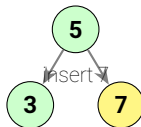
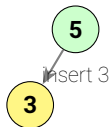
```
1  public class BST {
2      private TreeNode root;
3      public void insert(int value) {
4          root = insert(root, value);
5      }
6
7      private TreeNode insert(TreeNode node,
8                              int value) {
9          if (node == null) return new TreeNode(
10             value);
11          if (value < node.value) node.left
12             = insert(node.left, value);
13          else if (value > node.value) node.right
14             = insert(node.right, value);
15          // equal: duplicate, ignore (or update)
16          return node;
17      }
18  }
```

Pattern: return the (possibly new) subtree root

Each recursive call returns the node -- so the parent can wire its **left/right** pointer. At the empty slot, we return a new **TreeNode**. On the way back up, all assignments are no-ops.

BST Insertion -- Step by Step

Insert: 5, 3, 7, 1, 4



Each insert: follow the BST property down until you find a **null** slot

BST -- search()

```
1  public boolean search(int value) {
2      return search(root, value);
3  }
4
5  private boolean search(TreeNode node, int
    value) {
6      if (node == null) return false;  // not
        found
7      if      (value < (Integer) node.value)
        return search(node.left,  value);
8      else if (value > (Integer) node.value)
        return search(node.right, value);
9      else
        return true;  // found!
10 }
```

Complexity

- **Balanced BST:** $O(\log n)$ -- halve search space each step
- **Skewed BST:** $O(n)$ -- degenerates to linear scan

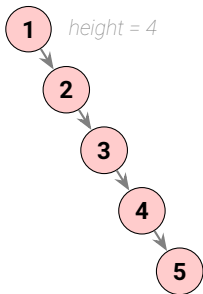
Best case vs. worst case

A balanced tree has height $\approx \log_2 n$.

A skewed tree has height $n - 1$.

The Skewed Tree Problem

Inserting **sorted** input: 1, 2, 3, 4, 5



A skewed BST is just a LinkedList!

Inserting sorted data always appends to the rightmost path. Height = $n - 1$. Search becomes $O(n)$.

Solution: self-balancing trees

AVL trees and **Red-Black trees** automatically rebalance on insert/delete. `java.util.TreeMap` uses a Red-Black tree internally, guaranteeing $O(\log n)$ always. See Appendix for details.

Map: key-based lookup

Problem shift

Linked structures are **position**-based: you walk to find index *i*.

A **Map** is **key**-based: you search by **key**, then read/write the corresponding *value*.

Core operations

- **put(key, value)**: insert or overwrite
- **get(key)**: retrieve value (or “not found”)
- **containsKey(key)**: membership by key
- **remove(key)**: delete by key

Hashing: key -> bucket index

Idea

We turn a key into an array index using a hash function:

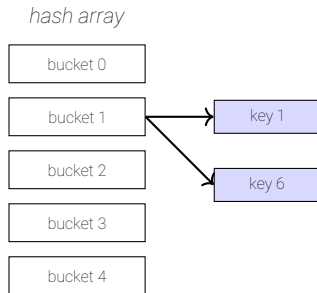
$$\text{index} = \text{abs}(\text{hash}(\text{key})) \% \text{capacity}$$

Then we only search in that bucket.

Collision

Different keys can produce the same index.

Those keys share one bucket.



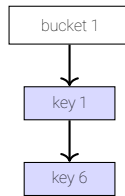
Example with capacity=5: $\text{hash}(1)=1$ and $\text{hash}(6)=1$
 $\Rightarrow$ collision in bucket 1.

Collision handling: chaining

What a bucket contains

To handle collisions, each bucket stores a **chain** of entries:

- each entry stores **key**, **value**, and a **next** pointer
- buckets[index] points to the first entry in that chain
- to find a key, walk the chain and compare keys



Example: key 1 and key 6 collide, so both live in bucket 1.

Build a simple HashMap (chaining)

```
1 // Pseudocode for HashMap with chaining:
2
3 put(key, value):
4     i = index(key)
5     walk buckets[i]:
6         if (cur.key == key) cur.value = value; return
7     buckets[i] = new Entry(key, value, next=buckets[i])
8
9 get(key):
10     i = index(key)
11     walk buckets[i]:
12         if (cur.key == key) return cur.value
13     return null
14
15 remove(key):
16     i = index(key)
17     prev = null; cur = buckets[i]
18     walk cur:
19         if (cur.key == key) unlink from chain; return
```

Lookup steps

- compute bucket index from the key
- walk the bucket chain until key matches
- put: overwrite or insert at head; get: return value/null; remove: unlink

Runtime intuition

Short chains => avg $O(1)$. Long chains => worst $O(n)$.

Complexity + resizing

Runtime intuition

Hashing keeps buckets spread out:

- short chains => average $O(1)$ for put/get
- long chains (many collisions) => worst $O(n)$

Keep it fast: resize

Use a **load factor**:

$$\text{load} = \text{size} / \text{capacity}$$

If it becomes too large:

- increase capacity
- rehash all entries into the new bucket array

Ordered map via a tree (TreeMap idea)

```
1 // BST rule: left keys smaller, right keys larger
2 get(key):
3     cur = root
4     while cur != null:
5         if key < cur.key: cur = cur.left
6         else if key > cur.key: cur = cur.right
7         else return cur.value
8     return null
9
10 put(key, value):
11     root = put(root, key, value)
12
13 put(node, key, value):
14     if node == null:
15         return new Node(key, value)
16     if key < node.key: node.left = put(node.left, key, value)
17     else if key > node.key: node.right = put(node.right, key, value)
18     else node.value = value // overwrite
19     return node
```

Why it stays ordered

With the BST rule (left keys smaller, right keys larger), an **inorder** traversal visits keys in sorted order.

Complexity

Balanced tree => height $O(\log n)$ => put/get $O(\log n)$. Skewed tree => height $O(n)$.

Set: membership (no duplicates)

What is a set?

A **set** is an unordered collection of *distinct* elements: you can test **membership**, **add**, and **remove** - each element appears at most once.

Unlike a **list**, there is no **index** or fixed order: adding the same element again does not create a second entry.

Versus a map

A **map** associates each **key** with a **value**. A **set** only records whether each element is **present** - no separate value per element. The defining rule is the same uniqueness maps enforce on **keys**.

Core operations

- **contains(x)**: is **x** in the set?
- **add(x)**: insert if absent (no duplicate)
- **remove(x)**: delete if present

Set backed by a map (uniqueness)

```
1  class IntSet {  
2      SimpleHashMap map; // key -> dummy  
3  
4      boolean contains(int x) {  
5          return map.get(x) != null;  
6      }  
7  
8      boolean add(int x) {  
9          boolean existed = contains(x);  
10         map.put(x, 1); // dummy value  
11         return !existed;  
12     }  
13  
14     void remove(int x) {  
15         map.remove(x);  
16     }  
17 }
```

Set semantics via key existence

A set is just *no duplicates*: an element exists iff its **key** exists in the backing map.

Object keys

For object keys, hashing works only if **equal keys map to equal buckets**. That depends on the **hashCode()** and **equals()** contract.

hashCode() & equals(): HashMap needs both

Two-step lookup

1. **hashCode()** → find the bucket (array index)
2. **equals()** → find the exact entry within the bucket

What happens with collisions?

Two keys can have the same hash (hash collision). HashMap stores them in the same bucket as a linked list. **equals()** is then used to find the right one.

The contract

Equal objects MUST have equal hashCodes.

`a.equals(b) == true`

⇒ `a.hashCode() == b.hashCode()`

The reverse need not hold: unequal objects *should* have different hashCodes (but are not required to).

What is a `hashCode`?

An integer on every object

Every Java **Object** provides `int hashCode()`.

HashMap uses it to choose a **bucket** index;

equals then finds the exact entry.

- **Not a unique id**: many objects *may* share the same `hashCode`; collisions are expected and handled.
- **Default on Object**: `hashCode` is identity-based (think “which object”), not derived from your fields unless you override.

Why keys care

If you define equality by **fields** (e.g. two **Points** with the same `x,y`) but leave the default `hashCode`, two equal keys can still land in **different buckets** - `get` returns `null`.

Implementing hashCode()

Rules of thumb

```
1  public class Point {
2      int x, y;
3
4      @Override
5      public int hashCode() {
6          return Objects.hash(x, y); // same
              fields as in equals
7      }
8  }
9
10 // HashMap keys: two equal Point instances
    find the same bucket
11 Point p1 = new Point(1, 2);
12 Point p2 = new Point(1, 2);
13 map.put(p1, "treasure");
14 IO.println(map.get(p2)); // "treasure"
```

- **hashCode** must combine the **same fields** that determine **equals**.
- **Use** `Objects.hash(...)` for a correct mix of fields (and **null-safe**).
- Add this whenever you override **equals**; IDEs can generate it.

Common mistake

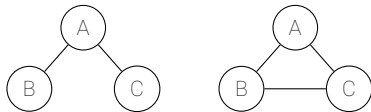
Skipping **hashCode** leaves `Object`'s identity-based hash---**HashMap** breaks **get** for equal-but-distinct

Graphs: when the structure can have cycles

Trees vs. graphs

A **tree** is **connected** and **acyclic**: exactly one simple path between any two nodes. A **graph** is **vertices** and **edges**; edges may close **cycles**---the model is not limited to a single root or a hierarchy.

- Examples: dungeon rooms and doors, waypoints on a map, quest or tech trees, multiplayer lobby matchmaking
- Typical questions: reachability, shortest paths (unweighted), connectivity



Left: tree (no cycle). Right: same three nodes with an extra edge---a **cycle**.

Directed vs. undirected (paths)

Undirected graph

Each edge is a **two-way** link: if $\{u, v\}$ is an edge, you can move from u to v and from v to u .

Directed graph

Edges have a **direction** from u to v (one-way unless you also add an edge from v to u).

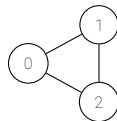
- **Path**: sequence of vertices connected by edges (respecting direction if directed)
- **Cycle**: path that starts and ends at the same vertex (length at least 1)

Representation: adjacency matrix

Idea

n vertices $\Rightarrow$ an $n \times n$ table. Entry `adj[u][v]` tells whether edge (u, v) exists.

- Undirected: often use `boolean[][]` or
`adj[u][v] = adj[v][u] = true`
- Space: $\Theta(n^2)$ ---fine when the graph is **dense** (many edges)



Triangle on $\{0, 1, 2\}$: three edges, matrix has three symmetric true pairs off-diagonal.

Representation: adjacency list

Idea

For each vertex v , store an **array of neighbor indices**: $\text{adj}[v]$ lists all w with an edge $v-w$.

- In Java: `int[] [] adj` where `adj.length == n` and `adj[v].length` is the **degree** of v
- Space: $\Theta(n + m)$ for m edges (each edge stored twice if undirected)---good for **sparse** graphs

Example

Triangle $0-1-2-0$:

`adj[0] = {1, 2}`

`adj[1] = {0, 2}`

`adj[2] = {0, 1}`

Matrix vs. list: when to use which

Adjacency matrix

- Test “is (u, v) an edge?” in $O(1)$
- Wastes space if $m \ll n^2$

	0	1	2
0			
1			
2			

Sketch: 3×3 table for 3 vertices.

Adjacency list

- Iterate neighbors of v in $O(\deg(v))$
- Dominant choice for large sparse graphs



Same graph: three neighbor lists instead of a full matrix.

Visited: why graphs differ from trees

In a tree

Following **left/right** from the root, you cannot accidentally revisit the parent unless you pass a reference---traversals are guided by structure.

In a graph

From vertex **v** you may have several neighbors; an edge can lead back to an already-seen vertex $\Rightarrow$ **infinite loops** if you do not remember where you have been.

Pattern

```
boolean[] visited = new  
boolean[n];  
visited[v] = true when you first  
discover v.
```

Breadth-first search (BFS)

Idea

Explore in **layers**: all vertices at distance 1, then 2, then 3, ... from a **source**---same spirit as level-order on a tree, but with a **visited** array.

- Use a **FIFO queue** of vertex indices (**int**)
- Dequeue **v**, enqueue each unvisited neighbor, mark visited

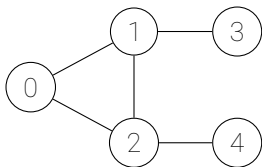
Data structures (no generics)

FIFO of vertex indices: **int[]** + **head/tail**, or linked queue from Session 5; **boolean[n]** visited.

BFS: pseudocode

```
1  // adj: adjacency list; n: vertices. FIFO via int[] ring (no generics).
2  void bfs(int start, int[][] adj, int n) {
3      boolean[] visited = new boolean[n];
4      int[] q = new int[n];
5      int head = 0, tail = 0;
6      visited[start] = true;
7      q[tail++] = start;
8      while (head < tail) {
9          int v = q[head++];
10         // visit v (e.g. IO.println(v))
11         for (int i = 0; i < adj[v].length; i++) {
12             int w = adj[v][i];
13             if (!visited[w]) {
14                 visited[w] = true;
15                 q[tail++] = w;
16             }
17         }
18     }
```

BFS: example (order from source 0)



Layers from 0

- Distance 0: {0}
- Distance 1: {1, 2}
- Distance 2: {3, 4}

Queue sketch (FIFO)

Start: enqueue 0. Then process 1,2; then 3,4.

Visit order (one valid BFS order): 0, 1, 2, 3, 4.

What BFS gives you

- **Unweighted shortest path (in edges)** from **start** to every reachable vertex: BFS discovers vertices in nondecreasing distance from the source
- **Reachability / connectivity** from a source: every vertex marked **visited** is in the same **connected component** as **start** (undirected case)
- **Time** with adjacency list: $O(n + m)$ ---each vertex dequeued once, each edge examined at most twice (undirected)

Depth-first search (DFS)

Idea

From **v**, go to one unvisited neighbor, recurse (or push on a stack), **backtrack** when stuck---same spirit as preorder on a tree, but **visited** prevents cycles.

- Implementation: **recursion** (call stack) or explicit **stack of int**

BFS vs. DFS

BFS: layers, shortest edge-count distances.

DFS: deep probes; often simpler to code recursively.

DFS: pseudocode (recursive)

```
1  boolean[] visited;
2
3  void dfsVisit(int v, int[][] adj) {
4      visited[v] = true;
5      // visit v
6      for (int i = 0; i < adj[v].length; i++) {
7          int w = adj[v][i];
8          if (!visited[w]) {
9              dfsVisit(w, adj);
10         }
11     }
12 }
13
14 void dfsFrom(int start, int[][] adj, int n) {
15     visited = new boolean[n];
16     dfsVisit(start, adj);
17 }
```

Undirected cycle (sketch)

Idea

When exploring **v**, if neighbor **w** is already **visited** and **w** is **not** the parent you came from, you have found a **cycle** (for simple undirected graphs, no self-loops).

- Track **parent[v]** while doing DFS, or pass parent into **dfsVisit(v, parent)**

DFS roles

Explore components, detect cycles, topological sort on **DAGs** (directed)---that variant comes with a different rule set.

Directed graphs & DAGs (outlook)

- **Directed** graphs: only follow edges in their direction; BFS/DFS still work with **visited**, but “shortest path” and connectivity need directed definitions
- **DAG**: directed acyclic graph---used for task dependencies; **topological order** (not covered here in detail)
- Session on **state machines** revisits graphs as labeled directed graphs naturally

Preview to next time:

Collections with **type parameters** make adjacency lists pleasant (**List** of neighbors per vertex); the **algorithms** stay the same.

What We Built Today

Binary Trees

- **TreeNode**: value + left + right
- **height()**: $O(n)$ recursion
- Inorder = sorted (BST)
- Preorder = copy/serialize
- Postorder = delete/eval
- Level-order uses Queue

BST

- $\text{Left} < \text{node} \leq \text{right}$
- **insert()**: recursive, return node
- **search()**: $O(\log n)$ balanced, $O(n)$ skewed
- Sorted insert $\rightarrow$ skewed
- TreeMap = balanced BST

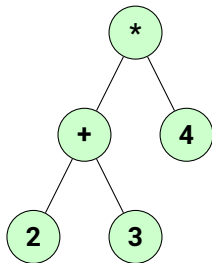
Maps & Sets

- HashMap: $O(1)$, no order
- TreeMap: $O(\log n)$, sorted
- HashSet/TreeSet: no values
- **hashCode** + **equals**: always override both

Optional: Graphs (if covered)

Appendix: Expression Tree Evaluation

An **expression tree** stores operators at internal nodes and operands at leaves. Postorder evaluation: operands first, then apply operator.



Postorder: **2, 3, +, 4, *** → evaluate left-to-right with a Stack: push 2, push 3, see + → pop 3, pop 2, push $2 + 3 = 5$, push 4, see * → pop 4, pop 5, push $5 * 4 = 20$.

Result: $(2 + 3) \times 4 = 20 \checkmark$

Appendix: AVL Trees (Conceptual)

Self-balancing BST

An **AVL tree** maintains the **balance factor** at each node:

$$\text{balance}(n) = \text{height}(n.\text{left}) - \text{height}(n.\text{right})$$

If $|\text{balance}(n)| > 1$ after an insert/delete, the tree performs a **rotation** to restore balance.

Four rotation cases

- **LL**: single right rotation
- **RR**: single left rotation
- **LR**: double rotation (left then right)
- **RL**: double rotation (right then left)

TreeMap uses Red-Black Trees

Red-Black trees are less strictly balanced but cheaper to maintain. Both guarantee $O(\log n)$.

Appendix: LinkedHashMap

```
1  import java.util.LinkedHashMap;
2
3  // LinkedHashMap: HashMap + insertion-order iteration
4  Map<String, Integer> cache = new LinkedHashMap<>();
5  cache.put("first", 1);
6  cache.put("second", 2);
7  cache.put("third", 3);
8
9  // Iterates in insertion order (not random like HashMap):
10 for (String key : cache.keySet()) {
11     IO.println(key); // first / second / third
12 }
```

When to use LinkedHashMap

- You need **$O(1)$** lookup **and** predictable iteration order
- LRU cache: subclass and override **`removeEldestEntry()`** to evict old entries
- Slightly more memory overhead than **`HashMap`** (extra doubly-linked list)

Appendix: Comparable vs. Comparator for Java's TreeMap

TreeMap needs an ordering

TreeMap sorts keys. The key type must either:

- **Implement Comparable** (natural order) -- e.g., `String`, `Integer`
- **Or supply a Comparator** at construction time

Example: reverse order

```
new TreeMap<>(Comparator.reverseOrder())
new TreeMap<>(Comparator.comparing(
    Student::getGrade))
```

Note on type parameters

The generics behind **Map<K, V>** -- and the formal explanation of all angle-bracket syntax -- are covered in **Generics & Collections Framework**.

HashMap needs neither

HashMap uses **hashCode**. No ordering. That is why insertion order is not preserved.